

Adding an RTOS to your MKR1000: a self-guided course

Christoph Ungricht

Contents

1	Adding an RTOS to your MKR1000: a self-guided course	1
1.1	Target facts you'll keep referring to	1
1.2	Course map	2
1.3	Module 1 -- What an RTOS is, and what changes	3
1.4	Module 2 -- Choosing an RTOS for Cortex-M0+	3
1.5	Module 3 -- Tasks, stacks, and context switching on Cortex-M	5
1.6	Module 4 -- The scheduler	7
1.7	Module 5 -- Synchronisation primitives	8
1.8	Module 6 -- (Optional) Write a 200-line cooperative scheduler from scratch	9
1.9	Module 7 -- FreeRTOS on the MKR1000: project layout	9
1.10	Module 8 -- Your first two tasks	13
1.11	Module 9 -- A producer/consumer with a queue	14
1.12	Module 10 -- A UART driver that plays nicely with the scheduler	15
1.13	Module 11 -- Common pitfalls and patterns	16
1.14	Module 12 -- What to integrate next	17
1.15	Appendix A -- Glossary	17
1.16	Appendix B -- <code>FreeRTOSConfig.h</code> field reference	18
1.17	Appendix C -- Practical comparison: the same program in five RTOSes	19

1 Adding an RTOS to your MKR1000: a self-guided course

A second course for the same board. Prerequisite: the bare-metal course ([guide/README.md](#)) up to Module 11 -- you must already have your own linker script, startup, working blink, SysTick, and UART. An RTOS is built *on top of* exactly the things you wrote there.

The philosophy is unchanged: Socratic, hint-driven, you write the code, you verify against datasheets and the RTOS source.

1.1 Target facts you'll keep referring to

- **MCU:** ATSAM21G18A, Cortex-M0+, 256 KiB flash, 32 KiB SRAM.
- **Documents added for this course:**
 1. **Cortex-M0+ Devices Generic User Guide** ([docs/Cortex-M0/dui0662a_...pdf](#)) -- you'll re-read the exception model chapter (PendSV, SVCall, SysTick) in earnest.
 2. **FreeRTOS Kernel source tree** (you'll vendor it) -- the actual code is the documentation. Repo: [FreeRTOS/FreeRTOS-Kernel](#) on GitHub.

3. **"Mastering the FreeRTOS Real Time Kernel"** -- the official free PDF book by Richard Barry. The reference for FreeRTOS API and concepts.
4. **AAPCS** (ARM Procedure Call Standard) -- relevant for understanding what context-switching actually has to save and restore.

Action 0: skim Chapter 3 ("Exceptions and Interrupts") of `dui0662a` again. Specifically, find the descriptions of **SVC**all and **PendSV**, the **Main Stack Pointer (MSP)** vs **Process Stack Pointer (PSP)**, and what happens **automatically** when an exception is taken on Cortex-M0+ (the "exception stack frame"). Write down which registers the hardware pushes and which it does not. This is the entire foundation of context switching.

1.2 Course map

Part	Module	Topic
I. Concepts	1	What an RTOS is, and what changes vs. the super-loop
	2	Choosing an RTOS for Cortex-M0+ (FreeRTOS, ChibiOS, NuttX, Zephyr, ThreadX, others)
II. How it works	3	Tasks, stacks, and context switching on Cortex-M
	4	The scheduler: priorities, preemption, cooperative vs. preemptive
	5	Synchronisation primitives: semaphores, mutexes, queues, event groups
III. A minimal scheduler from scratch (optional)	6	Write a 200-line cooperative scheduler yourself
IV. FreeRTOS on the MKR1000	7	Project layout, vendoring the kernel, <code>FreeRTOSConfig.h</code>
	8	First two tasks: blink + heartbeat
	9	A producer/consumer with a queue
	10	A UART driver that plays nicely with the scheduler
V. Going further	11	Common pitfalls, idle hook, low power
	12	What to integrate next: CMSIS-RTOS API, WiFi, tickless idle
Appendices	A	Glossary
	B	<code>FreeRTOSConfig.h</code> field reference
	C	Practical comparison: the same program in five RTOSes

1.3 Module 1 -- What an RTOS is, and what changes

You've been writing **super-loop** firmware: `while(1) { do_stuff(); }` with interrupts handling things that can't wait. This is simple and fast, and it's the right answer surprisingly often. An RTOS adds something the super-loop fundamentally cannot: **multiple independent control flows that appear to run in parallel, each blocking on its own conditions, without interfering with the others' timing.**

1.3.1 1.1 Mental model

An RTOS turns one CPU into the *illusion* of many CPUs, each running one "task" (a.k.a. thread). At any instant only one task is actually executing; the **scheduler** decides which. A task that is waiting on a timer, a queue, a semaphore, or an I/O event is *blocked* and consumes no CPU until the event happens.

1.3.2 1.2 What you give up

- **Determinism becomes harder to reason about**, because where you used to have one code path you now have N. Worst-case latency now depends on every task in the system and on the scheduler.
- **Memory**: each task needs its own stack. With 32 KiB total RAM on the SAMD21, every task's stack budget matters.
- **Complexity in shared resources**: as soon as two tasks touch the same variable or peripheral, you have to think about mutual exclusion.

1.3.3 1.3 What you gain

- **Blocking I/O reads naturally**: `xQueueReceive(q, &item, portMAX_DELAY)` is a one-line replacement for hand-rolled state machines.
- **Time as a first-class concept**: `vTaskDelay(pdMS_TO_TICKS(100))` is not a busy-wait; the scheduler runs other tasks during the 100 ms.
- **Priority-based response**: a high-priority task can preempt a low-priority one within microseconds when its event arrives.
- **Modularity**: each task is self-contained and locally reasonable.

1.3.4 1.4 When you do *not* want an RTOS

- Single-purpose hard-real-time control loops where one timer ISR is the whole program.
- Tiny memory budgets (sub-8 KiB SRAM) where two task stacks won't fit.
- Code where every cycle is accounted for and the scheduler overhead is intolerable. (FreeRTOS context switch on M0+ is roughly low-hundreds of cycles -- usually fine.)

Action: write a one-paragraph honest answer to "Does my MKR1000 project actually need an RTOS?" before you continue. If your blink + WiFi server from the bare-metal course is the goal, the answer might be "no, but learning it is worth doing anyway."

1.4 Module 2 -- Choosing an RTOS for Cortex-M0+

There are dozens. These are the ones you would realistically consider on this class of part.

RTOS	License	Footprint on M0+	SAMD21 support today	Notes
FreeRTOS	MIT (since v10.0)	very small (~5--10 KiB ROM, depends on features)	Generic Cortex-M0 port works directly	The default. Massive community, the book, vendor backing (AWS). What this course uses.
ChibiOS/RT	GPL + commercial exception	very small (~5 KiB ROM minimum)	Has SAMD21 HAL bindings in the project; verify current state	Famously fast and compact. Tighter integration of OS + HAL. Steeper learning curve than FreeRTOS.
Zephyr	Apache 2.0	medium-to-large	Has SAMD21 boards (e.g. <code>atsamd21_xpro</code>); MKR1000 not in-tree last I checked	Linux Foundation project. Huge scope: drivers, networking, BLE. Probably overkill for 32 KiB SRAM but worth knowing.
NuttX	Apache 2.0	medium	Has Atmel SAMD2x support	POSIX-style API (looks like Linux). Used in PX4 autopilot. Good if you want <code>pthread_create</code> muscle memory to transfer.
RIOT OS	LGPL 2.1	small-to-medium	Has SAMD21 boards	IoT-focused, modular, multi-threaded.
Azure RTOS ThreadX	MIT (since 2024)	very small, very fast	Generic Cortex-M0+ port; SAMD21 not a tier-1 board	Now Microsoft-stewarded, open source. Used on the Mars helicopter. Excellent technically; community is smaller than FreeRTOS's.
Mbed OS	Apache 2.0	medium	Was supported; ARM has wound down active development	Skip for new projects.

RTOS	License	Footprint on M0+	SAMD21 support today	Notes
embOS (Segger)	commercial	very small	Generic Cortex-M0+ port	Proprietary. Excellent. Free for evaluation.
Apache Mynewt	Apache 2.0	small	Some SAM support	Lesser known; designed for BLE/IoT.

1.4.1 2.1 What "support" really means here

For any of these on a non-tier-1 board, you'll do some work:

1. The kernel itself is **CPU-portable** -- it has a port for ARMv6-M (Cortex-M0/M0+), so the scheduler, context switch, and tick interrupt are already written for you.
2. The **board/chip-specific glue** (clock setup, GPIO, UART driver, `FreeRTOSConfig.h` tuning) is what you provide. **You already wrote most of this in the bare-metal course.**

So "MKR1000 not officially supported" is *not* a blocker for FreeRTOS or ChibiOS or ThreadX: the CPU port is what matters, and that exists. You bring your own bare-metal foundation.

1.4.2 2.2 Recommendation for this course: FreeRTOS

- Most mature ARMv6-M port.
- The free book ("Mastering the FreeRTOS Real Time Kernel") is one of the best embedded-systems books, period.
- API has remained stable for over a decade -- transfer to other projects is guaranteed.
- Plain C, no fancy build system, drop-in source tree.
- Small enough to walk through end to end.

After completing this course, *then* try a 10-task toy on ChibiOS or ThreadX as a comparative exercise. The concepts transfer.

1.5 Module 3 -- Tasks, stacks, and context switching on Cortex-M

Before installing an RTOS, understand exactly what it has to do at the machine level. This module is mostly reading the Cortex-M0+ user guide and drawing pictures. There is no code to write yet.

1.5.1 3.1 What "a task" actually is

A task is, at the hardware level:

1. A **stack** in RAM, owned by that task.
2. A **set of saved register values** representing what the task's CPU state will be when it resumes -- stored at the top of that task's stack.
3. A **Task Control Block (TCB)** holding metadata: priority, state (ready/blocked/running), pointer to top of saved stack, what it's waiting for.

That's it. The "task" abstraction is a stack + saved registers + a struct. The scheduler is the code that switches which stack is "the active one" and reloads its saved registers into the CPU.

1.5.2 3.2 What Cortex-M0+ hands you for free

On exception entry (any interrupt, including SysTick and PendSV), the hardware **automatically pushes 8 words** onto the current stack:

```
xPSR (program status)
PC   (return address)
LR   (link register)
R12
R3
R2
R1
R0
```

The currently-active stack pointer (MSP or PSP) is used. The hardware loads LR with a special EXC_RETURN value that tells the core, on the matching BX LR, which stack to pop from and whether to use MSP or PSP afterwards.

Action: in dui0662a, find the table of EXC_RETURN values for M0+. There are only a few. Write down what each one means (handler vs. thread mode, MSP vs. PSP).

The hardware **does not** save R4--R11 on exception entry. If your exception handler clobbers them, *it* has to save them. This is the half the scheduler has to do manually.

1.5.3 3.3 MSP and PSP

Cortex-M0+ has two stack pointers:

- **MSP** (Main Stack Pointer): used at reset and inside exception handlers.
- **PSP** (Process Stack Pointer): an alternate stack pointer, available to thread mode (non-handler code).

The standard RTOS convention: **the kernel and ISRs use MSP; each task uses PSP**. When the scheduler switches tasks, it changes which task's stack the PSP points to.

Action: find the instruction(s) that read/write MSP and PSP. (Hint: MRS and MSR with special-register names.) Also find how to switch the active stack pointer for thread mode (a bit in CONTROL register).

1.5.4 3.4 PendSV: the context-switch trampoline

Why context switches happen in PendSV specifically:

- It's a software-triggered exception. The scheduler sets a bit in SCB->ICSR and the core takes the PendSV exception at the next appropriate moment.
- It's set to the **lowest priority** in the system. That means PendSV fires only when no other interrupt is pending. **You never context-switch in the middle of someone else's ISR** -- you wait for it to finish.
- The actual context switch is therefore: ISR finishes -> PendSV runs -> PendSV saves the outgoing task's R4--R11, switches PSP to the incoming task's stack, restores R4--R11, returns -> hardware pops the incoming task's exception frame -> incoming task is now running.

1.5.5 3.5 SysTick: the heartbeat

The OS tick is exactly the SysTick you already met in bare-metal Module 10. Each tick:

1. SysTick ISR fires (preempting whatever task is running).
2. Increments a tick counter.
3. Checks: did any blocked task's delay just expire? Did any sleeping task's deadline arrive? Is a higher-priority task now ready?
4. If a switch is needed, the SysTick ISR sets the PendSV-pending bit.
5. Returns. On return, PendSV is now pending and (because it's lowest priority) fires immediately, doing the actual switch.

Two-stage design (SysTick decides, PendSV switches) keeps SysTick fast and lets it nest with other interrupts cleanly.

1.5.6 3.6 SVCall: the "start the scheduler" trampoline

SVC is the only practical way to enter handler mode from thread mode at will. FreeRTOS uses SVC exactly once: in `vTaskStartScheduler()`, to bootstrap the first task. The SVC handler "fakes" a return from an exception by manually setting up the first task's exception frame on its stack, then doing `BX LR` with an `EXC_RETURN` that pops to PSP. After that, the kernel never needs SVC again on M0+ -- all switching is PendSV.

Output of this module on paper: - A diagram of one task's stack with the saved frame at top. - A flowchart of "SysTick fires while Task A is running, and Task B is now higher-priority-ready" -- list every step from SysTick entry to Task B's first instruction executing. - The exact contents of an exception stack frame on M0+.

1.6 Module 4 -- The scheduler

Three flavours; an RTOS usually does all three at once depending on configuration.

1.6.1 4.1 Cooperative

Tasks run until they explicitly **yield** (`taskYIELD()`) or block. - Pro: zero race conditions on shared data within a task's run -- no preemption can interrupt it. - Con: one bad task hangs the whole system. - FreeRTOS supports this mode (`configUSE_PREEMPTION = 0`).

1.6.2 4.2 Preemptive priority-based

Every task has a priority. The scheduler always runs the **highest-priority ready task**. When a higher-priority task becomes ready (e.g. its semaphore was given by an ISR), the current task is preempted immediately.

- This is FreeRTOS's default and what you'll use.
- Tasks at the **same priority** time-slice on the tick if `configUSE_TIME_SLICING = 1`.

1.6.3 4.3 Round-robin (time-sliced) at equal priority

Within one priority level, tasks share the CPU in fixed time slices (one tick by default). Useful for "background" tasks that should run fairly but don't need priority ordering.

1.6.4 4.4 Idle task

When no task is ready to run, the scheduler runs a special **idle task** at priority 0. Its job is to (a) free memory of deleted tasks, and (b) call an **idle hook** function you supply -- the natural place to put

`__WFI()` to save power (see bare-metal Module 10).

1.6.5 4.5 Why priorities go wrong: priority inversion

Classic gotcha: high-priority task H blocks on a mutex held by low-priority task L; a medium-priority task M (that doesn't touch the mutex) preempts L and runs indefinitely; H is now effectively waiting on M. Result: H runs at M's priority, defeating the priority system.

Fix: **priority inheritance** -- while L holds a mutex H is waiting on, L temporarily inherits H's priority. FreeRTOS mutexes support this (plain semaphores do not -- use mutexes for resource protection).

Action: read FreeRTOS book Chapter 7 ("Mutual Exclusion"). It is the chapter most likely to save you from a real bug.

1.7 Module 5 -- Synchronisation primitives

The primitives FreeRTOS provides and when to use which.

Primitive	Use case	Notable property
Binary semaphore	"Event happened, wake the waiter."	ISR-safe <code>xSemaphoreGiveFromISR</code> . No priority inheritance.
Counting semaphore	Pool of N resources; gate N concurrent users.	Increment up to a max.
Mutex	Mutual exclusion on a shared resource.	Priority inheritance. Not ISR-safe.
Recursive mutex	Same task can take the mutex multiple times.	Slightly more expensive.
Queue	Pass data between tasks/ISRs, FIFO.	Copy-by-value. ISR-safe variants.
Stream buffer	Variable-length byte streams (e.g. UART RX).	Single-reader, single-writer.
Message buffer	Like stream buffer but message-delimited.	
Event group	24 boolean flags; tasks wait on combinations.	"Wait for flags 1 AND 3" or "1 OR 3".
Task notification	Lightweight per-task signalling.	Lower overhead than a semaphore; usually the right tool.

Rule of thumb: **task notifications first, queues second, semaphores third, mutexes only for shared resources**. This minimises overhead and bug surface.

1.7.1 5.1 ISR safety

Every primitive has two API forms: one for tasks, one for ISRs (suffixed `FromISR`). ISR variants do not block -- they return "higher-priority task woken?" through a parameter. At the end of your ISR you call `portYIELD_FROM_ISR(woken)` to request a context switch on the way out if needed.

Action: in FreeRTOS source, open `queue.c` and find `xQueueGenericSendFromISR`. Read it end to end. It's about 60 lines and shows you exactly how an RTOS ISR-safe API is constructed.

1.8 Module 6 -- (Optional) Write a 200-line cooperative scheduler from scratch

This is the most educational module in the course and the most skippable. Allocate a weekend. By the end you will *never again* be confused about how an RTOS works.

1.8.1 6.1 Spec for your toy RTOS

- Up to N tasks (`#define MAX_TASKS 4`), each with a fixed-size stack (e.g. 256 bytes).
- Cooperative only: tasks call `yield()` to give up the CPU.
- One-tick delay: `delay(ticks)` that yields until ticks have elapsed, driven by your existing SysTick.
- No priorities (round-robin).

1.8.2 6.2 What you have to write

1. A **TCB struct** holding stack pointer and "wake at tick" value.
2. A **scheduler function** that picks the next runnable task.
3. **task_create(fn, stack, size)** that initialises a TCB and writes a fake exception-stack-frame onto the task's stack (so the first time PendSV "restores" the task, the CPU pops into your task function).
4. **yield()** that triggers PendSV.
5. The **PendSV handler in assembly** that saves R4--R11 onto the outgoing PSP, switches PSP to the incoming task's saved SP, restores R4--R11, returns.
6. **scheduler_start()** that uses SVCall (or equivalent) to switch into PSP and execute the first task.

1.8.3 6.3 Hints for the tricky parts

- Faking a stack frame: at task creation, push onto the task's stack the values { `xPSR=0x01000000`, `PC=task_fn`, `LR=task_exit_stub`, `R12=0`, `R3=0`, `R2=0`, `R1=0`, `R0=arg` } -- in reverse order so they're popped correctly. Then push 8 more zeros for R4--R11. Save the resulting SP in the TCB.
- `xPSR=0x01000000` sets the Thumb bit; without it, you'll fault on the first instruction.
- `task_exit_stub` is a function that loops forever -- "what to do if a task returns" -- because pop-into-task is a one-way trip.

Action: get one task running. Then two. Then `delay()` between them. When two LEDs blink at different rates from two real tasks on one CPU, you have understood the most important idea in this entire course.

If you skip this module, you can still use FreeRTOS productively. But the magic-feeling of context switching never quite disappears.

1.9 Module 7 -- FreeRTOS on the MKR1000: project layout

Time to use the real thing.

1.9.1 7.1 Get the kernel source

```
git clone https://github.com/FreeRTOS/FreeRTOS-Kernel
```

You only need this one repo -- the "FreeRTOS" mega-repo with demos and add-on libraries is huge and you don't need 99% of it for learning.

1.9.2 7.2 Vendor it into your project

```
mkr1000/
|-- ...your bare-metal files from the first course...
|-- freertos/
|   |-- kernel/           <- git submodule or vendored copy of FreeRTOS-Kernel
|   |   |-- include/
|   |   |-- portable/GCC/ARM_CM0/    <- the port you'll compile
|   |   `-- ...
|   `-- FreeRTOSConfig.h <- YOUR config, lives in YOUR project, not the kernel
```

The kernel is a frozen dependency; your `FreeRTOSConfig.h` is project code.

1.9.3 7.3 What to compile

From the kernel tree, add to your Makefile:

- `tasks.c`, `queue.c`, `list.c`, `timers.c`, `event_groups.c`, `stream_buffer.c` (in the kernel root).
- `portable/GCC/ARM_CM0/port.c` and `portable/GCC/ARM_CM0/portmacro.h` include path.
- Either `portable/MemMang/heap_1.c` (simplest, no free) or `heap_4.c` (best general-purpose). Pick `heap_4` for this course.

The kernel `include/` directory goes into `-I`.

Action: get the kernel compiling against your existing bare-metal build. It should produce a few object files and not link yet -- you haven't called any FreeRTOS functions.

1.9.4 7.4 FreeRTOSConfig.h essentials

This is your tuning file. Copy a template from `FreeRTOS-Kernel/examples/template_configuration/FreeRTOSConf` and adapt. The fields that **must** match your hardware:

- `configCPU_CLOCK_HZ` = your CPU frequency (8 MHz at boot, 48 MHz after Module 11 of the bare-metal course).
- `configTICK_RATE_HZ` = 1000 (1 ms ticks is standard).
- `configMINIMAL_STACK_SIZE` = 128 words (512 bytes); tune later.
- `configTOTAL_HEAP_SIZE` = some fraction of your 32 KiB SRAM, e.g. (8 * 1024). **Tasks come out of this heap.**
- `configMAX_PRIORITIES` = 5 to start.
- `configUSE_PREEMPTION` = 1.
- `configUSE_IDLE_HOOK` = 1 (we'll use it for `__WFI()`).
- `configCHECK_FOR_STACK_OVERFLOW` = 2 (paranoid).
- `configUSE_MALLOC_FAILED_HOOK` = 1.

Appendix B lists every field with notes.

1.9.5 7.5 The three handlers FreeRTOS needs from your vector table

In your `startup.s` vector table, the slots for **SVC**all, **PendSV**, and **SysTick** must point to FreeRTOS's handlers, not your `Default_Handler`. The names are:

```
vPortSVCHandler
xPortPendSVHandler
xPortSysTickHandler
```

The simplest way: add weak aliases in `startup.s`:

```
.weak SVC_Handler
.thumb_set SVC_Handler, vPortSVCHandler

.weak PendSV_Handler
.thumb_set PendSV_Handler, xPortPendSVHandler

.weak SysTick_Handler
.thumb_set SysTick_Handler, xPortSysTickHandler
```

Then have your vector table reference `SVC_Handler` etc., which the port's symbols override.

Action: build and link. The link should succeed. If it fails on undefined `SVC_Handler`, your vector-table symbol name doesn't match -- pick one convention and stick to it.

1.9.6 7.6 Priorities and interrupts

Cortex-M0+ has only 4 priority levels (2 bits). Two FreeRTOS rules follow:

1. **PendSV must be lowest priority.** The port code does this for you in `xPortStartScheduler()`.
2. **Any ISR that calls *FromISR APIs must have a priority numerically greater than or equal to `configMAX_SYSCALL_INTERRUPT_PRIORITY`.** On Cortex-M0/M0+ this is implicit (the port masks all interrupts inside critical sections); on M3+ you have to set the priority explicitly. **On M0+, every ISR may call FromISR APIs.**

1.9.7 7.7 Static vs dynamic allocation

FreeRTOS can allocate every object (task TCBs and stacks, queues, mutexes, semaphores, timers, the idle task, the timer task) **dynamically** from `heap_4.c` (what Modules 7-10 use), or **statically** at compile/link time (no heap involved). Pick one model on purpose; production embedded almost always wants the static one.

1.9.7.1 Why care

- **Determinism:** no `malloc` failures at runtime. All memory accounted for at link time; the linker map tells you exactly how much RAM the kernel consumes. `configTOTAL_HEAP_SIZE` stops being a guessing game because the heap can be zero.
- **Smaller image:** skip `heap_4.c` entirely; save several KiB.
- **Certifiability:** safety-critical guidelines (MISRA, IEC 61508) prefer no dynamic allocation post-init. Many regulated codebases mandate it.
- **No fragmentation:** irrelevant for create-once-at-startup patterns but matters if you ever delete/recreate tasks.

Trade-off: you write a tiny bit more boilerplate per object (a static stack buffer, a static TCB), and you must provide two callbacks the idle task and timer task use to find their own stacks.

1.9.7.2 Enabling it In `FreeRTOSConfig.h`:

```
#define configSUPPORT_STATIC_ALLOCATION 1
#define configSUPPORT_DYNAMIC_ALLOCATION 0 /* optional; saves a bit more */
```

You then must supply two callbacks anywhere in your firmware:

```

#include "FreeRTOS.h"
#include "task.h"
#include "timers.h"

/* Idle task storage. */
static StaticTask_t   idle_tcb;
static StackType_t    idle_stack[configMINIMAL_STACK_SIZE];

void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer,
                                   StackType_t **ppxIdleTaskStackBuffer,
                                   uint32_t      *pulIdleTaskStackSize)
{
    *ppxIdleTaskTCBBuffer    = &idle_tcb;
    *ppxIdleTaskStackBuffer = idle_stack;
    *pulIdleTaskStackSize    = configMINIMAL_STACK_SIZE;
}

/* Timer task storage (only needed if configUSE_TIMERS = 1). */
static StaticTask_t   timer_tcb;
static StackType_t    timer_stack[configTIMER_TASK_STACK_DEPTH];

void vApplicationGetTimerTaskMemory(StaticTask_t **ppxTimerTaskTCBBuffer,
                                    StackType_t **ppxTimerTaskStackBuffer,
                                    uint32_t      *pulTimerTaskStackSize)
{
    *ppxTimerTaskTCBBuffer    = &timer_tcb;
    *ppxTimerTaskStackBuffer = timer_stack;
    *pulTimerTaskStackSize    = configTIMER_TASK_STACK_DEPTH;
}

```

Without these two functions and `configSUPPORT_STATIC_ALLOCATION = 1`, the kernel won't link.

1.9.7.3 Creating tasks and queues statically For each task and kernel object, declare its storage and use the `*Static` variant of the create call.

```

static StaticTask_t blink_tcb;
static StackType_t  blink_stack[256];

static StaticQueue_t led_queue_storage;
static uint8_t       led_queue_buffer[8 * sizeof(led_cmd_t)];

int main(void)
{
    /* ...inits... */

    QueueHandle_t led_queue =
        xQueueCreateStatic(8, sizeof(led_cmd_t),
                           led_queue_buffer, &led_queue_storage);

    TaskHandle_t blink_h =
        xTaskCreateStatic(blink_task, "blink", 256, NULL, 1,

```

```

        blink_stack, &blink_tcb);

    /* ... other tasks ... */

    vTaskStartScheduler();
    for (;;)
}

```

Notice nothing returns NULL because of allocation failure -- the storage exists by virtue of being declared. The static variants cannot fail.

1.9.7.4 When to use which

- **Dynamic (xTaskCreate, heap_4.c):** learning, prototyping, hobby projects, when you genuinely create/delete tasks at runtime. This guide's Modules 8-10 use it for brevity.
- **Static (xTaskCreateStatic):** production firmware, safety-critical code, anything that has to be predictable across reboots and that you don't want to debug under memory pressure. Switch to this **before** you ship.

Mixed-mode is supported (both flags = 1); use it during migration if needed.

Action: take your two-task blink from Module 8 and convert it to fully static allocation. Compare flash and RAM usage in the linker map file before and after.

1.10 Module 8 -- Your first two tasks

Replace your bare-metal main() with:

```

#include "FreeRTOS.h"
#include "task.h"

static void blink_task(void *arg) {
    (void)arg;
    for (;;) {
        gpio_toggle(LED_PIN);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

static void heartbeat_task(void *arg) {
    (void)arg;
    for (;;) {
        uart_puts("alive\r\n");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

int main(void) {
    clocks_init_48mhz();    // from your bare-metal Module 11
    uart_init(115200);
    gpio_init();
}

```

```

xTaskCreate(blink_task,      "blink", 128, NULL, 1, NULL);
xTaskCreate(heartbeat_task, "hb",     256, NULL, 1, NULL);

vTaskStartScheduler();
for (;;) // unreachable
}

void vApplicationIdleHook(void) { __asm volatile ("wfi"); }

```

Two tasks at the same priority, time-slicing. Both should produce their respective outputs concurrently.

Action: build, flash, observe. If the LED blinks but UART is silent, the UART driver is being called from a task context that the kernel hasn't fully prepared -- usually a sign of a stack too small. Bump heartbeat's stack to 512 and try again.

1.10.1 8.1 Common first-time failures

- **HardFault on vTaskStartScheduler:** SVCcall vector not pointing at vPortSVCHandler. Check the vector table.
- **Tasks never run, system hangs:** SysTick vector wrong, or configCPU_CLOCK_HZ doesn't match the actual CPU frequency, or you forgot to call your clock init before the scheduler.
- **malloc failed immediately:** configTOTAL_HEAP_SIZE is too small -- task stacks come out of the heap when allocated dynamically.
- **Random crashes after running a while:** stack overflow. Enable configCHECK_FOR_STACK_OVERFLOW = 2 and supply vApplicationStackOverflowHook.

1.11 Module 9 -- A producer/consumer with a queue

The blink + heartbeat duo doesn't really need a kernel. The first thing that *does* is two tasks communicating.

1.11.1 9.1 Spec

- Task A reads a button (PORT input, optionally with an EIC interrupt on edge).
- Task B drives the LED in patterns depending on what Task A sends.
- A FreeRTOS queue carries enum messages from A to B.

1.11.2 9.2 Sketch

```

typedef enum { CMD_BLINK_FAST, CMD_BLINK_SLOW, CMD_OFF } led_cmd_t;

static QueueHandle_t led_queue;

static void button_task(void *arg) {
    bool prev = false;
    for (;;) {
        bool now = gpio_read(BUTTON_PIN);
        if (now && !prev) {
            led_cmd_t c = CMD_BLINK_FAST;
            xQueueSend(led_queue, &c, portMAX_DELAY);
        }
    }
}

```

```

        prev = now;
        vTaskDelay(pdMS_TO_TICKS(20)); // simple debounce
    }
}

static void led_task(void *arg) {
    led_cmd_t cmd = CMD_OFF;
    TickType_t period = pdMS_TO_TICKS(1000);
    for (;;) {
        if (xQueueReceive(led_queue, &cmd, period) == pdTRUE) {
            switch (cmd) {
                case CMD_BLINK_FAST: period = pdMS_TO_TICKS(100); break;
                case CMD_BLINK_SLOW: period = pdMS_TO_TICKS(500); break;
                case CMD_OFF:         period = portMAX_DELAY;      break;
            }
        }
        gpio_toggle(LED_PIN);
    }
}

int main(void) {
    /* ...inits... */
    led_queue = xQueueCreate(4, sizeof(led_cmd_t));
    xTaskCreate(button_task, "btn", 256, NULL, 2, NULL);
    xTaskCreate(led_task,    "led", 256, NULL, 1, NULL);
    vTaskStartScheduler();
    for (;;)
}

```

Notice: `xQueueReceive` blocks with a timeout that doubles as the LED's toggle period. The task is asleep when nothing is happening, which is exactly the point.

Variation: do the button via EIC interrupt, and have the ISR call `xQueueSendFromISR`. This is the canonical "ISR talks to task" pattern.

1.12 Module 10 -- A UART driver that plays nicely with the scheduler

Your bare-metal UART driver almost certainly busy-waits on the TX "data register empty" flag. Inside an RTOS this wastes the CPU -- during the ~87 us it takes to send one byte at 115200 baud, *some other task could be running*.

1.12.1 10.1 The pattern

- Hold a **stream buffer** (or queue) of bytes pending TX.
- `uart_puts(str)` writes into the stream buffer; if it's full, blocks.
- A **UART TX interrupt** fires when the data register is empty; the ISR pulls one byte from the stream buffer and writes it; if the buffer is empty, disables the TX interrupt.
- `uart_puts` enables the TX interrupt to kick off the chain.

For RX, symmetrically: - A **UART RX interrupt** fires when a byte arrives; the ISR pushes it into a stream buffer. - `uart_getc()` (a task-context call) blocks on `xStreamBufferReceive`.

1.12.2 10.2 Why a stream buffer

The FreeRTOS **stream buffer** is specifically designed for single-writer + single-reader byte streams -- exactly the UART case. Lighter than a queue.

1.12.3 10.3 What to be careful about

- The TX ISR runs at *some* hardware priority. On M0+ you don't have to worry about masking; just write the ISR using `*FromISR` APIs and call `portYIELD_FROM_ISR` at the end.
- Don't call `uart_puts` from an ISR. If you must log from an ISR, use a separate "log queue" and have a dedicated task drain it via the UART. ISRs should be short.

Action: rewrite your bare-metal UART driver with this pattern. Time `uart_puts` and confirm it returns quickly even for long strings -- the bytes drain in the background.

1.13 Module 11 -- Common pitfalls and patterns

In rough order of how often you'll trip on them:

1. **Stack too small.** FreeRTOS task stacks are tiny by default (`configMINIMAL_STACK_SIZE` words, often $128 = 512$ bytes). A task that calls `printf` or `snprintf` blows this instantly. Fix: enable `configCHECK_FOR_STACK_OVERFLOW = 2`, supply `vApplicationStackOverflowHook`, and grow stacks until the hook stops firing.
2. **`vTaskDelay(1)` is not "1 ms".** It's "delay until the next tick" -- could be 0--1 ms in practice. Use `vTaskDelay(pdMS_TO_TICKS(N))`.
3. **Forgetting `portYIELD_FROM_ISR`.** Your ISR woke a task but the scheduler doesn't know to run it until the next tick. Always:

```
BaseType_t woken = pdFALSE;
xSomethingFromISR(..., &woken);
portYIELD_FROM_ISR(woken);
```
4. **Calling task APIs from an ISR.** `vTaskDelay`, `xQueueSend` (non-FromISR) and friends will assert or hardfault. Use `*FromISR` variants.
5. **Heap fragmentation.** Avoid creating/deleting tasks at runtime. Prefer to create all tasks at startup. With `heap_4` and stable allocation patterns, fragmentation is rarely an issue; with create/delete churn it's a guarantee.
6. **Priority assignment as "1 is best".** In FreeRTOS, **higher number = higher priority**. Idle is 0. This is the opposite of some other RTOSes (e.g. classic Unix `nice`). Read this twice when designing your priorities.
7. **Using a mutex from an ISR.** Mutexes are not ISR-safe (no priority inheritance possible from a non-task context). Use a binary semaphore if an ISR needs to signal a task.
8. **Two-tier interrupt mistake:** forgetting that on M0+, *all* interrupts can call `*FromISR` APIs (unlike M3+ where you have to compare priority to `configMAX_SYSCALL_INTERRUPT_PRIORITY`). Don't over-engineer this on M0+; just write the ISRs.

1.13.1 11.1 Idle hook + low power

Set `configUSE_IDLE_HOOK = 1` and provide:


```
void vApplicationIdleHook(void) {
    __asm volatile ("wfi");
}
```

The CPU sleeps whenever no task is ready. Combined with a slower tick rate (e.g. 100 Hz instead of 1 kHz), this can knock idle current down by an order of magnitude. For *real* low power, look into FreeRTOS's **tickless idle** (Module 12).

1.14 Module 12 -- What to integrate next

- **CMSIS-RTOS v2 API**: a vendor-neutral wrapper over FreeRTOS (and others). Worth knowing because lots of vendor sample code uses it, but FreeRTOS native API is more direct.
- **Tickless idle** (`configUSE_TICKLESS_IDLE`): stop SysTick during long idles, drive wakeups from a low-power timer. Significant power savings; significant subtlety.
- **The WiFi side from the bare-metal course (Module 14)**: port the WINC1500 BSP so SPI transfers block the calling task on a stream buffer instead of busy-waiting. Now the WINC's slow operations don't stall everything else.
- **A web server task**: turn the WiFi server into a task that uses blocking `recv()` and a queue of work. The single-task hand-coded HTTP server from the bare-metal course becomes a clean producer/ consumer with multiple sockets.
- **Software timers**: `xTimerCreate` for periodic and one-shot callbacks that don't justify a whole task.

This is also the right moment to add a real **logging framework** (a task that drains a stream buffer to UART, with severity levels and timestamps) and a **command-line interface** task that reads UART RX and dispatches commands. Both are 100-line tasks once you have the primitives.

1.15 Appendix A -- Glossary

- **AAPCS** -- ARM Procedure Call Standard. Defines which registers a callee may clobber (r0--r3, r12) vs. must preserve (r4--r11), so the scheduler knows what to save.
- **Context switch** -- saving one task's registers and restoring another's so the CPU appears to switch threads.
- **EXC_RETURN** -- the special LR value loaded by hardware on exception entry; on return, encodes which mode/stack to pop to.
- **Hook** -- a callback function the RTOS calls at well-defined points (idle, tick, stack overflow, malloc failure) that you provide.
- **Idle task** -- the always-runnable task at priority 0, runs when nothing else can.
- **ISR-safe** -- of an API: callable from interrupt context.
- **MSP** -- Main Stack Pointer. The default SP, used by handler mode.
- **PendSV** -- Pendable Service exception. Software-triggered, lowest-priority, used as the context-switch trampoline.
- **Priority inheritance** -- temporary priority boost given to a mutex-holding task to prevent priority inversion.
- **Priority inversion** -- bug where a high-priority task is effectively delayed by a medium-priority task because a low-priority task holds a needed resource.
- **PSP** -- Process Stack Pointer. The alternate SP, used by tasks in thread mode under an RTOS.
- **RTOS** -- Real-Time Operating System. An OS whose primary design goal is bounded, predictable timing.

- **Scheduler** -- the code that picks the next task to run.
- **SVCall (SVC)** -- supervisor-call exception, used to bootstrap the scheduler.
- **Task** -- a thread of execution with its own stack and saved context. The unit the scheduler operates on.
- **Task notification** -- lightweight per-task signal; lower overhead than a semaphore.
- **TCB** -- Task Control Block. The struct holding one task's metadata.
- **Tick** -- the periodic SysTick interrupt that gives the kernel its time base.
- **Tickless idle** -- mode where the tick is suppressed during long idle periods to save power.
- **Yield** -- voluntary scheduler invocation (`taskYIELD()`); the current task lets others run.

1.16 Appendix B -- FreeRTOSConfig.h field reference

Not exhaustive -- the FreeRTOS documentation has the full list. These are the fields you'll touch on the MKR1000.

Field	Meaning	Suggested value here
<code>configUSE_PREEMPTION</code>	Preemptive scheduler	1
<code>configCPU_CLOCK_HZ</code>	Actual CPU clock in Hz	match your clock init
<code>configTICK_RATE_HZ</code>	Tick frequency	1000
<code>configMAX_PRIORITIES</code>	Number of priority levels	5
<code>configMINIMAL_STACK_SIZE</code>	Idle task stack in words (not bytes)	128
<code>configTOTAL_HEAP_SIZE</code>	Heap for tasks/queues/etc.	(8 * 1024) to start
<code>configMAX_TASK_NAME_LEN</code>	Bytes for task names	10
<code>configUSE_16_BIT_TICKS</code>	Use <code>uint16_t</code> for tick counts	0 (32-bit ticks are fine)
<code>configIDLE_SHOULD_YIELD</code>	Idle yields to equal-priority tasks	1
<code>configUSE_TASK_NOTIFICATIONS</code>	Enable notification API	1
<code>configUSE_MUTEXES</code>	Enable mutex APIs	1
<code>configUSE_RECURSIVE_MUTEXES</code>	Enable recursive mutexes	0 (rarely needed)
<code>configUSE_COUNTING_SEMAPHORES</code>	Enable counting semaphores	1
<code>configQUEUE_REGISTRY_SIZE</code>	Number of named queues for debugger	8
<code>configUSE_TIME_SLICING</code>	Round-robin at equal priority	1
<code>configUSE_IDLE_HOOK</code>	Call <code>vApplicationIdleHook</code>	1
<code>configUSE_TICK_HOOK</code>	Call <code>vApplicationTickHook</code> every tick	0 initially
<code>configCHECK_FOR_STACK_OVERFLOW</code>	1/2; 2 is paranoid	2
<code>configUSE_MALLOC_FAILED_HOOK</code>	Call hook on heap exhaustion	1
<code>configRECORD_STACK_HIGH_ADDRESS</code>	Helps with stack inspection	1
<code>configUSE_TRACE_FACILITY</code>	Adds trace info per task	1 (small cost)
<code>configGENERATE_RUN_TIME_STATS</code>	Per-task CPU usage	0 until you need it
<code>configUSE_TIMERS</code>	Software timer task	1
<code>configTIMER_TASK_PRIORITY</code>	Priority of timer task	one above idle to start
<code>configTIMER_QUEUE_LENGTH</code>	Outstanding timer commands	5
<code>configTIMER_TASK_STACK_DEPTH</code>	Stack for timer task in words	<code>configMINIMAL_STACK_SIZE</code> * 2
<code>INCLUDE_vTaskDelay</code>	API-inclusion macros (one per func)	enable what you call

Field	Meaning	Suggested value here
INCLUDE_xTaskGetSchedulerState		1
configKERNEL_INTERRUPT_PRIORITY	M0+: leave as port default	--
configMAX_SYSCALL_INTERRUPT_PRIORITY	FreeRTOS: irrelevant on M0+; defined on M3+	--

Action: once your two-task blink works, drop `configTOTAL_HEAP_SIZE` from 8 KiB to 4 KiB and see whether anything breaks. Then raise it back. You'll develop intuition for the heap budget.

1.17 Appendix C -- Practical comparison: the same program in five RTOSes

Module 2 compared RTOSes in a feature table. This appendix does it *operationally*: the same small program -- two threads, one queue -- written in each kernel's native API. Read this to feel where each kernel sits on the "minimal vs. opinionated" axis.

1.17.1 The program

Two threads. Thread A wakes every 500 ms and posts a 32-bit counter value onto a queue. Thread B blocks on the queue and toggles an LED each time a value arrives. (Equivalent in spirit to Module 9 of this course -- the producer/consumer.)

The thread bodies are identical in pseudocode:

```
producer():
    n = 0
    forever:
        sleep_ms(500)
        n += 1
        queue_send(q, n)

consumer():
    forever:
        v = queue_receive(q)
        led_toggle()
```

Only the API surface changes between kernels. Assume your bare-metal `led_toggle()` exists and works.

1.17.2 C.1 FreeRTOS (what this course uses)

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

static QueueHandle_t q;

static void producer(void *arg) {
    uint32_t n = 0;
    (void)arg;
    for (;;) {
```

```

        vTaskDelay(pdMS_TO_TICKS(500));
        n++;
        xQueueSend(q, &n, portMAX_DELAY);
    }
}

static void consumer(void *arg) {
    uint32_t v;
    (void)arg;
    for (;;) {
        xQueueReceive(q, &v, portMAX_DELAY);
        led_toggle();
    }
}

int main(void) {
    board_init();
    q = xQueueCreate(8, sizeof(uint32_t));
    xTaskCreate(producer, "p", 256, NULL, 1, NULL);
    xTaskCreate(consumer, "c", 256, NULL, 1, NULL);
    vTaskStartScheduler();
    for (;;)
}

```

- Dynamic allocation from a heap (heap_4.c).
- Tasks are void(void*), never return.
- pdMS_TO_TICKS converts ms to scheduler ticks.
- API prefix conventions: v = void, x = returns a status type, p = pointer.

1.17.3 C.2 ChibiOS/RT

```

#include "ch.h"
#include "hal.h"

static THD_WORKING_AREA(wa_producer, 256);
static THD_WORKING_AREA(wa_consumer, 256);

static mailbox_t mb;
static msg_t mb_buffer[8];

static THD_FUNCTION(producer, arg) {
    uint32_t n = 0;
    (void)arg;
    while (true) {
        chThdSleepMilliseconds(500);
        n++;
        chMBPostTimeout(&mb, (msg_t)n, TIME_INFINITE);
    }
}

static THD_FUNCTION(consumer, arg) {

```

```

    msg_t v;
    (void)arg;
    while (true) {
        chMBFetchTimeout(&mb, &v, TIME_INFINITE);
        led_toggle();
    }
}

int main(void) {
    halInit();
    chSysInit();
    chMBOBJECTInit(&mb, mb_buffer, 8);
    chThdCreateStatic(wa_producer, sizeof(wa_producer),
                     NORMALPRIO, producer, NULL);
    chThdCreateStatic(wa_consumer, sizeof(wa_consumer),
                     NORMALPRIO, consumer, NULL);
    chThdSleep(TIME_INFINITE);
}

```

- **Static** allocation by default: THD_WORKING_AREA reserves a stack at compile time.
- "Mailbox" instead of "queue"; carries msg_t (machine word).
- HAL is bundled with the kernel -- you typically use ChibiOS's HAL too, not your own.
- More macros, more compile-time configuration; smallest images.

1.17.4 C.3 Zephyr RTOS

```

#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

K_MSGQ_DEFINE(q, sizeof(uint32_t), 8, 4);

static void producer(void *a, void *b, void *c) {
    uint32_t n = 0;
    while (1) {
        k_msleep(500);
        n++;
        k_msgq_put(&q, &n, K_FOREVER);
    }
}

static void consumer(void *a, void *b, void *c) {
    uint32_t v;
    while (1) {
        k_msgq_get(&q, &v, K_FOREVER);
        led_toggle();
    }
}

K_THREAD_DEFINE(p_id, 512, producer, NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(c_id, 512, consumer, NULL, NULL, NULL, 5, 0, 0);

```

```
void main(void) { /* threads autostart */ }
```

- Threads and queues defined at **link time** with macros that emit static objects. No `xCreate` calls.
- Three opaque `void*` args per thread (Zephyr's calling convention).
- Priorities: **lower number = higher priority** (opposite of FreeRTOS).
- Threads with priority defined at `K_THREAD_DEFINE` time start automatically when the kernel starts.

1.17.5 C.4 Azure RTOS ThreadX

```
#include "tx_api.h"
```

```
#define STACK_BYTES 1024
```

```
static TX_THREAD producer_thread;
static TX_THREAD consumer_thread;
static uint8_t producer_stack[STACK_BYTES];
static uint8_t consumer_stack[STACK_BYTES];
```

```
static TX_QUEUE q;
static uint32_t q_storage[8];
```

```
static void producer_entry(ULONG arg) {
    uint32_t n = 0;
    (void)arg;
    while (1) {
        tx_thread_sleep(50); /* 50 ticks; default 10ms => 500ms */
        n++;
        tx_queue_send(&q, &n, TX_WAIT_FOREVER);
    }
}
```

```
static void consumer_entry(ULONG arg) {
    uint32_t v;
    (void)arg;
    while (1) {
        tx_queue_receive(&q, &v, TX_WAIT_FOREVER);
        led_toggle();
    }
}
```

```
void tx_application_define(void *first_unused_memory) {
    tx_queue_create(&q, "q", TX_1_ULONG, q_storage, sizeof(q_storage));
    tx_thread_create(&producer_thread, "p", producer_entry, 0,
                    producer_stack, STACK_BYTES, 10, 10,
                    TX_NO_TIME_SLICE, TX_AUTO_START);
    tx_thread_create(&consumer_thread, "c", consumer_entry, 0,
                    consumer_stack, STACK_BYTES, 10, 10,
                    TX_NO_TIME_SLICE, TX_AUTO_START);
}
```

```
int main(void) {
```

```

board_init();
tx_kernel_enter(); /* never returns */
}

```

- Queue items are sized in 32-bit words (TX_1_ULONG means each message is 1 word).
- Static stacks; you allocate them.
- All kernel objects created from tx_application_define, a callback invoked once by tx_kernel_enter.
- API verbs are very regular: tx_<object>_<action>. Reading the code aloud usually tells you what it does.
- Smallest of the mainstream ones; the cleanest API by most measures.

1.17.6 C.5 NuttX (POSIX-style)

```

#include <pthread.h>
#include <mqueue.h>
#include <unistd.h>

static mqd_t mq;

static void *producer(void *arg) {
    uint32_t n = 0;
    (void)arg;
    for (;;) {
        usleep(500000);
        n++;
        mq_send(mq, (const char *)&n, sizeof(n), 0);
    }
}

static void *consumer(void *arg) {
    uint32_t v;
    (void)arg;
    for (;;) {
        mq_receive(mq, (char *)&v, sizeof(v), NULL);
        led_toggle();
    }
}

int main(int argc, char *argv[]) {
    struct mq_attr a = { .mq_maxmsg = 8, .mq_msgsize = sizeof(uint32_t) };
    mq = mq_open("/q", O_RDWR | O_CREAT, 0666, &a);

    pthread_t pt, ct;
    pthread_create(&pt, NULL, producer, NULL);
    pthread_create(&ct, NULL, consumer, NULL);

    pthread_join(pt, NULL); /* never returns */
    return 0;
}

```

- POSIX threads + POSIX message queues. **Indistinguishable from user-space Linux code.**
- Power and weakness: if your Linux muscle memory is strong, NuttX needs no new vocabulary. But you've lost the "see exactly where every byte goes" property the other kernels give you.

1.17.7 C.6 Side-by-side observations

Aspect	FreeRTOS	ChibiOS	Zephyr	ThreadX	NuttX
Default allocation	Dynamic (heap)	Static	Static	Static (you provide stacks)	Dynamic
Priority direction	Higher num = higher prio	Higher num = higher prio	Lower num = higher prio	Lower num = higher prio	POSIX (1-99, higher = higher prio)
Thread "starting"	Explicit <code>xTaskCreate</code>	Explicit <code>chThdCreateStatic</code>	Implicit at link time via macros	Explicit <code>tx_thread_create</code> with <code>TX_AUTO_START</code>	Explicit <code>pthread_create</code>
Queue/mailbox carries	Arbitrary item size	Single <code>msg_t</code> (word)	Arbitrary item size	Multiples of <code>ULONG</code>	Arbitrary byte buffer
API verb style	<code>xQueueSend</code> , <code>vTaskDelay</code>	<code>chMBPost</code> , <code>chThdSleep</code>	<code>k_msgq_put</code> , <code>k_msleep</code>	<code>tx_queue_send</code> , <code>tx_thread_sleep</code>	<code>mq_send</code> , <code>pusleep</code>
HAL bundled?	No -- bring your own	Yes -- ChibiOS HAL	Yes -- device drivers + DT	No	Yes -- POSIX drivers
First-class config	<code>FreeRTOSConfig.h</code>	<code>chconf.h</code> + <code>halconf.h</code>	<code>Kconfig</code>	<code>tx_user.h</code>	<code>Kconfig</code>
LoC for this demo	~30	~30	~25	~35	~30

1.17.8 C.7 Which to pick when -- a take

- **FreeRTOS** if community size and book/docs matter most. Probably the right default if you don't have other constraints. *This is why this course uses it.*
- **ChibiOS** if you want the smallest binary, the cleanest static resource model, and the HAL bundled. Excellent on M0+.
- **Zephyr** if your project will grow into networking/BLE/USB at some point and you want one ecosystem to bring you there. Expect to learn Kconfig and devicetree (the Linux course's Module 5 concept applies).
- **ThreadX** if you appreciate small, regular APIs and don't need a huge community. Used for high-reliability projects (Microsoft's IoT line; the Mars helicopter).
- **NuttX** if your team has Linux/POSIX muscle memory and the project genuinely needs that API surface (mqueues, pthreads, /dev nodes).

Action: pick one alternative (ChibiOS or ThreadX recommended for M0+), port this same producer-consumer to it, and compare: - Binary size (`size -A` on the .elf). - RAM usage (your linker map file). - Worst-case latency from queue post to consumer LED toggle (use a GPIO toggle in the producer right before `*Send` and measure with a logic analyser). The numbers will surprise you in both directions.